# Securing APIs

## Lecture 9: Authentication and Authorisation
## Expanded explanations and guided hands-on activities

COMP3028 Software Construction

Western Sydney University — Surabaya Campus

# What students should be able to do

1. Explain why an API must protect both data and operations.
2. Distinguish identity checks from permission checks.
3. Hash and verify passwords with bcrypt.
4. Follow a session login and a JWT login through successive requests.
5. Test missing credentials, wrong permissions, object ownership, and token expiry.

# An API exposes a controlled way to use a system

- An application programming interface (API) defines how one program requests data or actions from another.
- In a web API, the client sends an HTTP request to a server endpoint such as `GET /notes/n1`.
- The server checks the request, performs the permitted operation, and returns a response, often JSON.
- An endpoint is the combination of a route and an HTTP method. Reading and creating notes are different operations.

# A request and its response

```
GET /notes/n1 HTTP/1.1
Host: example.invalid
Authorization: Bearer <access-token>

HTTP/1.1 200 OK
Content-Type: application/json

{"note":{"id":"n1","text":"My lecture notes"}}
```

The path selects the resource. The header carries a credential. The response status describes the outcome.

# Why secure APIs?

- **Protect sensitive data:** a student should see their own records, not another student's records.
- **Prevent unauthorised access:** a request must satisfy the endpoint's access policy.
- **Protect business logic:** ordinary users must not perform administrative operations.
- **Maintain trust and integrity:** attackers must not silently change data or impersonate users.

# Activity: identify the resource and the rule

1. Imagine a university API with `GET /results/u1`.
2. Name the resource: the results belonging to student `u1`.
3. Identify who may read it: that student and authorised staff.
4. Explain why knowing the URL is insufficient proof of permission.

**Checkpoint**

A safe endpoint checks both the caller's identity and their right to access this particular record.

# API attacks: interpreting the original lecture example

- The source lecture uses a Salt Security industry report to illustrate API attack activity.
- Treat vendor reports as evidence from a particular sample and reporting period. They do not measure every API worldwide.
- The practical lesson remains: exposed endpoints need deliberate security checks and monitoring.
- For this class, we focus on the three threat groups in the original lecture rather than reproducing an undated statistic.

# Three threat groups from the original lecture

| Threat | What goes wrong | Example defence |
|---|---|---|
| Injection | Input becomes executable query or command logic | Parameterised queries and safe APIs |
| Broken authentication | The server accepts an incorrect or stolen identity | Sound credential and token handling |
| Insecure endpoints | Access rules or transport protection are missing | HTTPS and server-side access checks |

## Scope

These are teaching categories, not a complete inventory of API vulnerabilities.

# Injection: data crosses into executable instructions

- **SQL injection:** concatenating untrusted text into a SQL statement can change what the database executes.
- **Command injection:** constructing a shell command from request parameters can allow unexpected commands.
- Validate type, length, and permitted values at the API boundary.
- Use parameter binding for SQL. Prefer library functions over executing a shell. Validation alone is insufficient.

# Mini-exercise: repair a database query

```
// Unsafe illustration only: do not run this pattern.
const sql = "SELECT id FROM users WHERE username='"
  + username + "'";

// Parameterised example for a PostgreSQL client:
const result = await db.query(
  'SELECT id FROM users WHERE username = $1',
  [username]
);
```

The placeholder keeps the username as data. This is a conceptual comparison; the class lab uses an in-memory array and does not need a database.

# Broken authentication: an identity check fails

- **Credential stuffing:** someone tries username/password pairs stolen from other services.
- **Session hijacking:** someone reuses a stolen session credential to impersonate its owner.
- **Weak token handling:** missing signature or expiry checks let invalid credentials through.
- Defences include HTTPS, login throttling, MFA where appropriate, and explicit token validation.
- OAuth 2.0 concerns delegated authorisation. OpenID Connect adds an identity layer for authentication.

# Insecure endpoints: a correct login is only one layer

- **Missing HTTPS:** credentials and responses can be exposed in transit.
- **Missing access checks:** a logged-in student can call an admin route.
- **Excessive response data:** the API returns password hashes or internal fields that the client never needs.
- An unguessable URL and a hidden button are not access controls. The server must enforce the policy on every request.

# Threat checkpoint

1. A server accepts a token without verifying its signature. Which threat group fits?
2. A database query includes user input through string concatenation. Which defence is needed?
3. A student changes the note ID and receives somebody else's note. Which rule is missing?

**Checkpoint**

Answers: broken authentication; a parameterised query; an object-level ownership or access check.

# Authentication answers: who is making this request?

- Authentication verifies a claimed identity using evidence called credentials.
- A username usually identifies an account. A password or another factor supplies evidence of control.
- After login, the client commonly presents a session ID or access token on later requests.
- Possession of a bearer credential can be enough to act as that user. Protect it like a temporary password.

# Credentials and authentication factors

| Factor | Example | What it demonstrates |
|---|---|---|
| Something you know | Password or PIN | Knowledge of a secret |
| Something you have | Security key or authenticator device | Possession of a device or key |
| Something you are | Fingerprint or face check | A biometric property |

API keys often identify an application or client. They do not automatically identify the human using it.

**Quick check**

A password plus a second password still uses one factor category.

# Password storage: what the database should contain

- Store a password hash produced by a password-hashing function, not the original password.
- At login, compare the submitted password with the stored hash using the library's verification function.
- A salt makes identical passwords produce different stored hashes. bcrypt includes salt and cost information in its output.
- We retain bcrypt to match the source lecture. OWASP generally prefers Argon2id for new systems; bcrypt has a 72-byte input limit.
- A fast general-purpose hash such as plain SHA-256 is unsuitable on its own for password storage.

# Step 1: prepare the tools

1. Install a supported Node.js LTS release, including npm, before class.
2. Use a code editor such as VS Code and an API client such as Postman.
3. Create a folder named `api-lab`. Open that folder in the editor.
4. Open the editor's terminal in that folder. Run the checks on the next slide.

**Checkpoint**

You need internet access for the first package installation. No database, cloud account, or paid API is required.

# Step 2: confirm Node.js and initialise the project

```
node --version
npm --version
npm init -y
npm install express@5 express-session@1 bcrypt@6 jsonwebtoken@9
```

Run one line at a time. Successful installation creates `node_modules` and a lockfile. Use filenames ending in `.mjs` so Node recognises `import`.

# The project files and how to use them

- `hash-demo.mjs`: the short password-hashing activity.
- `server.mjs`: the complete local API, assembled from numbered code slides in the appendix.
- `test.mjs`: an optional automated check supplied in the accompanying project.
- If using the supplied project folder, run `npm install` there and skip `npm init -y`.
- For manual entry, append every numbered server block in order to one file. Do not run a partial server file.

# Step 3: create hash-demo.mjs

```
import bcrypt from 'bcrypt';
const password = 'AliceDemo!42';
const hash1 = await bcrypt.hash(password, 10);
const hash2 = await bcrypt.hash(password, 10);
console.log('Hash 1:', hash1);
console.log('Hash 2:', hash2);
console.log('Same hashes?', hash1 === hash2);
console.log('Correct password?', await bcrypt.compare(password, hash1));
console.log('Wrong password?', await bcrypt.compare('wrong', hash1));
```

Save the file, then run `node hash-demo.mjs`. The asynchronous calls wait for bcrypt to finish.

# Step 4: interpret the hashing output

```
Hash 1: $2b$10$... (your output will vary)
Hash 2: $2b$10$... (your output will vary)
Same hashes? false
Correct password? true
Wrong password? false
```

The two hashes differ because bcrypt generates fresh salts. Changing the password to `wrong` must make verification fail. The cost 10 is a work-factor setting, not ten individual hash operations.

# Step 5: assemble and start the API

1. Create `server.mjs`. Copy the complete numbered appendix blocks in order, or open the supplied file.
2. Save the file. In the terminal, run `node server.mjs`.
3. Keep this terminal running. Requests will be sent from Postman.
4. To stop the server, press Ctrl+C. After an edit, stop it and run the command again.

**Checkpoint**

The terminal should display: `API ready at http://127.0.0.1:3000`.

# Step 6: make the first request in Postman

1. Choose a new HTTP request and select the method `GET`.
2. Enter `http://127.0.0.1:3000/health`.
3. Click Send. Look at the response status and JSON body.
4. Expect `200 OK` and `{"status":"ok"}`.

**Checkpoint**

If the connection fails, check that the server terminal is still running and that the address uses port 3000.

# The two fictional accounts

| | Alice | Bob |
|---|---|---|
| Username | `alice` | `bob` |
| Demo password | `AliceDemo!42` | `BobDemo!42` |
| User ID | `u1` | `u2` |
| Role | student | admin |
| Permissions | `notes:read` | `notes:read`, `notes:write` |
| Owned note | `n1` | `n2` |

Use these fictional passwords only for this local activity. The server hashes them at startup and returns only selected public user fields.

# Lab boundaries: understanding the simplifications

- The API binds to `127.0.0.1`. All data and sessions live in memory and disappear on restart.
- Random signing secrets are generated at startup. Restarting invalidates existing sessions and JWTs.
- HTTP and `secure:false` cookies are used only to make the localhost activity work.
- This demo omits persistent storage, rate limiting, MFA, and complete browser CSRF protection. It is a teaching application, not a deployment template.

# Reading Express code: request, response, and next

- An Express route has a method, a path, and functions that process a matching request.
- `req` contains incoming data: `req.body` for JSON, `req.params` for path values, and headers for credentials.
- `res` builds the outgoing response. For example, `res.status(401).json(...)` sends a JSON error.
- A middleware function performs a check before the final handler. Calling `next()` continues to the next function.
- When a check fails, `return res.status(...)` ends that path so the protected handler does not run.

# Tracing one protected request

```
app.get('/jwt/admin',
  requireJwt,
  requireRole('admin'),
  (req, res) => {
    res.json({ message: 'Admin access granted' });
  }
);
```

Read top to bottom: validate the token and set `req.user`; check the user's role; send the success response.

## Checkpoint

Missing credentials stop at the first check. Alice's valid token passes the first check but fails the role check.

# Authorisation answers: what may this caller do?

- Authentication establishes an identity. Authorisation applies a policy to an action and resource.
- Role-based access control (RBAC) groups permissions into roles such as student or admin.
- Permission-based access control assigns specific capabilities such as `notes:read` or `notes:write`. We use PBAC in the source lecture's sense.
- Object-level rules still matter: a user with read permission may only read their own note in our lab.

# Our access policy before writing tests

| Operation | Alice | Bob |
|---|---|---|
| View own profile | Allow | Allow |
| Open admin endpoint | Deny | Allow |
| Read own note | Allow | Allow |
| Read another user's note | Deny | Deny |
| Create a note | Deny | Allow |

## A deliberate policy choice

Admin access does not automatically bypass ownership checks. The application must explicitly define any such privilege.

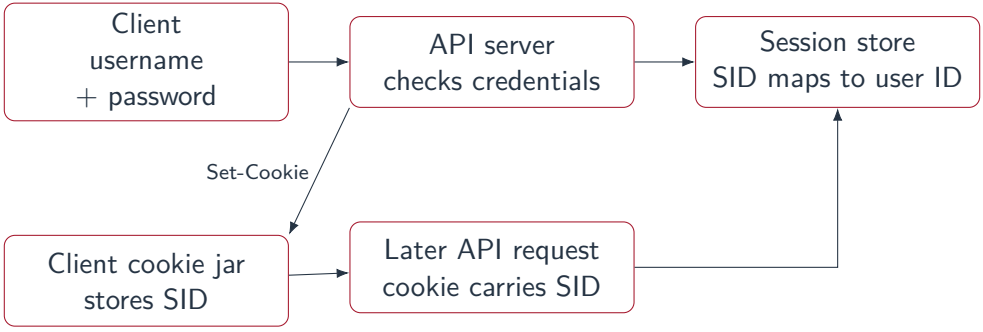
# HTTP status codes used in this lab

| Code | Meaning in the exercises |
|---|---|
| 200 | The request succeeded and has a response body. |
| 201 | A new note was created. |
| 204 | Logout succeeded with no response body. |
| 400 | The request input is invalid. |
| 401 | Acceptable authentication credentials are missing or invalid. |
| 403 | The authenticated user lacks the required role or permission. |
| 404 | The note is absent or not visible to this user. |

# Sessions maintain state across HTTP requests

- HTTP requests do not automatically remember a previous login.
- With a server-side session, the server stores session data and gives the client an opaque session ID.
- The client usually stores the session ID in a cookie. The server uses it to look up session data on later requests.
- In our lab, the session stores `userId`. Password hashes remain in the user records, outside the session.

# A session login and a later request




The cookie travels with eligible requests. The server sends `Set-Cookie` when it sets or updates a cookie, not necessarily on every response.

# Reading the session configuration

- `httpOnly:true` prevents browser JavaScript from reading the cookie directly. It does not remove every XSS risk.
- `sameSite:"strict"` restricts cross-site cookie sending. Browser applications still need a suitable CSRF defence.
- `secure:true` requires HTTPS. The lab uses `false` only on localhost HTTP.
- `saveUninitialized:false` avoids saving untouched anonymous sessions.
- Login regenerates the session ID to reduce session fixation risk. The default memory store is for this exercise only.

# Session activity 1: try a protected route before login

1. Create a GET request to `http://127.0.0.1:3000/session/me`.
2. Clear cookies for `127.0.0.1` in Postman if you previously used the lab.
3. Click Send without adding any login credentials.
4. Expect `401` with the error `Login required`.

**Checkpoint**

The health endpoint is public. The profile endpoint requires an authenticated session.

# Session activity 2: log in as Alice

In Postman, choose POST, then Body > raw > JSON. Paste only the JSON object into the body.

```
POST http://127.0.0.1:3000/session/login
Content-Type: application/json

{
  "username": "alice",
  "password": "AliceDemo!42"
}
```

Click Send. Expect 200 and Alice's public user fields. In response Headers, find `Set-Cookie`; the cookie name is `lab.sid`.

# Session activity 3: reuse the cookie

1. Leave the Postman cookie jar enabled. It should retain `lab.sid`.
2. Send GET to `http://127.0.0.1:3000/session/me` again.
3. Expect 200 and Alice's ID `u1`. No password is needed on this request.
4. Inspect Cookies to confirm that the credential is an opaque session value, not a JSON user profile.

**Checkpoint**

Always use the same host: cookies for 127.0.0.1 will not automatically apply to localhost.

# Session activity 4: logout and verify the result

1. Send POST to `http://127.0.0.1:3000/session/logout`. No body is needed.
2. Expect 204 with an empty response body.
3. Send GET to `/session/me` again using the full base address.
4. Expect 401. The server destroyed the stored session and cleared its cookie.

**Checkpoint**

Removing the client cookie and invalidating server-side session data are different actions. This logout endpoint does both.

# Session activity 5: test a failed login

1. Repeat POST `/session/login`, but change the password to `wrong`.
2. Expect `401` and `Invalid credentials`.
3. Change the password to the number `123`, without quotation marks. Expect `400`.
4. Explain the distinction: the wrong string fails authentication; a number fails input validation.

**Checkpoint**

The API never returns the stored password hash in a profile or login response.

# A JSON Web Token carries claims

- In this lecture, a JWT is a signed token in compact JWS form with three dot-separated parts: header, payload, and signature.
- The header describes the signing algorithm. The payload contains claims such as subject and expiry.
- The signature allows a verifier to detect changes when it uses a trusted key and approved algorithm.
- Base64url encoding is readable encoding. A signed JWT does not conceal its payload.
- Changing claims requires a newly signed token. Editing the text does not produce a valid credential.

# Reading the header and payload

```
// Header (decoded)
{"alg":"HS256","typ":"JWT"}

// Payload (illustrative times only)
{"sub":"u1","iss":"comp3028-lab",
 "aud":"comp3028-client","iat":1800000000,
 "exp":1800000300}
```

`sub`: subject; `iss`: issuer; `aud`: intended audience; `iat`: issue time; `exp`: expiry. The timestamp values are seconds since the Unix epoch.

# What signing and verification actually do

- Our lab uses HS256: HMAC-SHA-256 authenticates the encoded header and payload using a shared secret.
- With asymmetric signing, a private key signs and a corresponding public key verifies. These keys have different roles.
- Verification must check the algorithm, signature, expiry, issuer, and audience expected by the API.
- `jwt.decode()` reads a token. `jwt.verify()` validates it. Decoding alone cannot establish trust.
- Keep passwords, secrets, and unnecessary personal information out of JWT claims.

# JWT flow: from login to a protected request

1. The client submits a username and password to the login endpoint.
2. The server verifies the password and signs a short-lived access token.
3. The client sends the token in `Authorization: Bearer <token>`.
4. The server verifies the token, identifies the user, and checks access rules.
5. Only after these checks does the endpoint return the protected resource.

**Checkpoint**

The same verified identity can still receive 403 when it lacks the required permission.

# JWT activity 1: request Alice's token

1. Create POST `http://127.0.0.1:3000/jwt/login`.
2. Choose Body > raw > JSON and reuse Alice's login object from the session activity.
3. Click Send. Expect 200 with `accessToken`, `tokenType`, and `expiresIn`.
4. Copy only the token string, without surrounding quotation marks. Keep it available for the following requests.

**Checkpoint**

The token lifetime is five minutes (300 seconds). If it expires during class, log in again.

# JWT activity 2: call the profile endpoint

1. Create GET `http://127.0.0.1:3000/jwt/me`.
2. In the Authorization tab, select Bearer Token and paste Alice's token into the Token field.
3. Click Send. Expect 200 and Alice's public profile.
4. Select No Auth and send again. Expect `401`. Restore Bearer Token afterwards.

**Checkpoint**

Postman adds the word Bearer for you. Do not type Bearer into its Token field.

# JWT activity 3: inspect the payload locally

```
// Save as inspect-token.mjs, beside server.mjs.
import jwt from 'jsonwebtoken';
const token = process.argv[2];
console.log(jwt.decode(token, { complete: true }));

// Terminal command (replace the placeholder):
// node inspect-token.mjs "PASTE_YOUR_LAB_TOKEN"
```

Observe `sub`, `iss`, `aud`, and `exp`. This script deliberately decodes without verifying. It is for inspection only. Use only fictional tokens if trying the original lecture's jwt.io debugger.

# JWT activity 4: test token tampering

1. Keep an unchanged copy of Alice's token.
2. In Postman, change one character near the beginning of the signature, the third token segment.
3. Send GET `/jwt/me`. Expect `401` and `Invalid or expired token`.
4. Restore the unchanged, unexpired token. The same request should return 200.

**Checkpoint**

A modified token can still look like a JWT. Its appearance does not prove authenticity.

# JWT activity 5: observe expiration

1. For a shorter experiment, change `expiresIn: '5m'` in the server to `'15s'`.
2. Also change response field `expiresIn: 300` to 15. Save and restart.
3. Log in again to get a new token. Immediately call `/jwt/me`; expect 200.
4. Wait at least 16 seconds and send the same request. Expect `401`.
5. Restore both settings, restart, and obtain fresh tokens before continuing.

**Checkpoint**

Restarting changes the signing secret. Always obtain a new token after a restart.

# JWT logout differs from session logout

- Deleting a JWT from one client stops that client from sending it, but a copied token may remain valid until expiry.
- Immediate revocation requires additional state or policy, such as a denylist or a token version checked by the API.
- Our lab has no JWT logout endpoint or refresh-token flow. Re-login creates another short-lived access token.
- The JWT routes also look up the user in memory. They avoid a session lookup but are not a fully self-contained, stateless implementation.

# Access activity 1: prove that authentication is insufficient

1. Log in through `/jwt/login` as Alice and obtain a fresh token.
2. Send GET `http://127.0.0.1:3000/jwt/admin` using that token.
3. Expect 403. Alice has a valid identity but lacks the admin role.
4. Log in as Bob using `BobDemo!42`; repeat with Bob's token and expect 200.

**Checkpoint**

Switching identities means replacing the token. Editing a username in a GET request does not change the authenticated user.

# Access activity 2: test a fine-grained permission

Choose raw JSON for the body and Bearer Token for authorisation.

```
POST http://127.0.0.1:3000/jwt/notes
Authorization: Bearer <token>
Content-Type: application/json

{"text":"My security revision note"}
```

Alice: 403 because she lacks `notes:write`. Bob: 201 with the created note. Its owner is Bob. A fresh server assigns the first new note ID `n3`.

# Access activity 3: test object ownership

1. With Alice's token, GET `/jwt/notes/n1`. Expect 200.
2. Change only the note ID to `n2`. Expect 404.
3. With Bob's token, GET `/jwt/notes/n2`. Expect 200.
4. With Bob's token, GET `/jwt/notes/n1`. Expect 404.

**Checkpoint**

Both users have read permission, but the resource must also belong to the caller. The lab uses 404 to avoid confirming another user's note exists.

# Access activity 4: validate data and response fields

1. With Bob's token, POST `/jwt/notes` with body `{"text":""}`. Expect 400.
2. Repeat with `{"text":42}`. Expect 400.
3. Repeat with a short non-empty string. Expect 201.
4. Inspect a profile response and confirm that it contains no `passwordHash`.

**Checkpoint**

Authentication, authorisation, input validation, and response filtering solve different problems.

# Evidence students should record

| Test | Identity / input | Expected |
|---|---|---|
| Session profile before login | No session | 401 |
| Session profile after login | Alice session | 200 |
| Session profile after logout | Destroyed session | 401 |
| JWT profile | No token / modified token | 401 |
| JWT profile | Valid Alice token | 200 |
| Admin endpoint | Alice / Bob | 403 / 200 |
| Create note | Alice / Bob, valid body | 403 / 201 |
| Read note n2 | Alice / Bob | 404 / 200 |
| Create note | Bob, empty text | 400 |
| JWT profile after expiry | Expired token | 401 |

For each row, record the request, status, and a one-sentence explanation. Redact tokens in any submitted screenshots.

# A balanced comparison

| Aspect | Server-side session | Signed JWT access token |
|---|---|---|
| Client credential | Opaque session ID | Token containing claims |
| Server validation | Look up session data | Validate token and access policy |
| Scaling | Shared store supports multiple servers | Verification can avoid session-store lookups |
| Revocation | Destroy stored session | Extra state often needed before expiry |
| Client protection | Secure handling of session credential | Secure handling of token |
| Data exposure | Data stays in the session store | Claims readable unless separately encrypted |
| Operational cost | Store availability and cleanup | Key management and token lifecycle |

# Storage and transport choices apply to both approaches

- A session ID or JWT can be stolen if the client environment is compromised. Signing does not prevent replay of a stolen token.
- HttpOnly cookies reduce direct JavaScript access, while cookie-based authentication requires CSRF considerations.
- Browser localStorage is accessible to JavaScript and therefore exposed to token theft through XSS.
- HTTPS protects credentials in transit. It does not replace permission checks, safe storage, or expiry handling.
- Our API-client exercise uses cookies for sessions and an Authorization header for JWTs to make the difference visible.

# Which approach fits the application?

- For a browser application that needs immediate logout, a well-managed server-side session is often straightforward.
- For APIs with multiple independent token-verifying services, signed access tokens can be useful.
- Sessions can scale with a shared store. JWTs are not automatically faster or safer in every architecture.
- Choose using revocation needs, client type, infrastructure, and operational complexity. Neither choice removes authorisation rules.

# Troubleshooting the hands-on activities

| Symptom | Check / correction |
|---|---|
| `node` not recognised | Install Node.js and reopen the terminal. |
| Cannot find a package | Run `npm install` inside the correct project folder. |
| Import syntax error | Save as `.mjs`, not `.js.txt`. |
| Connection refused | Start the server and verify port 3000. |
| `EADDRINUSE` | Stop your earlier lab server before starting another. |
| Session stays at 401 | Enable cookies and use 127.0.0.1 consistently. |
| JWT suddenly returns 401 | Re-login after expiry or a server restart. |
| JSON gives 400 | Use raw JSON, quoted keys, and no trailing comma. |
| Unexpected 403 | Check the token belongs to the intended account. |

# From classroom demo to a deployed application

- Use HTTPS, managed secrets, and a production session store. Plan key rotation and expiration.
- Add login throttling, abuse monitoring, and appropriate MFA. Avoid logging passwords or bearer tokens.
- Use persistent user storage and enforce access policies consistently on every route.
- For browsers, design CSRF and XSS defences alongside cookie and token choices.
- Keep dependencies maintained and test both allowed and denied requests. Do not publish this local demo unchanged.

# Exit questions

1. Why can two bcrypt hashes differ even when the password is identical?
2. Why does Alice get 403 on the admin endpoint after a successful login?
3. Why does reading the JWT payload not prove that the token is valid?
4. Why is removing a JWT from a client different from destroying a session?
5. What check prevents Alice from reading Bob's note?

**Checkpoint**

Suggested answers: random salts; missing role; no signature validation; copied JWTs may survive until expiry; object ownership.

# Appendix: assembling the exact server

- The following numbered blocks form one complete file: `server.mjs`.
- Copy the blocks in order without their slide titles. Keep blank lines and braces.
- Do not add a second `app.listen()` or duplicate imports.
- After the last block, save and run `node server.mjs`. Return to the first request activity.
- The supplied project contains the same code and an optional automated regression check.

# Server code 1/10

Append to `server.mjs` in this order.

```
// Local teaching demo. Do not deploy this in-memory example.
import express from 'express';
import session from 'express-session';
import bcrypt from 'bcrypt';
import jwt from 'jsonwebtoken';
import { randomBytes } from 'node:crypto';

const app = express();
app.use(express.json({ limit: '10kb' }));
const sessionSecret = randomBytes(32).toString('hex');
const jwtSecret = randomBytes(32).toString('hex');
app.use(session({
  name: 'lab.sid', secret: sessionSecret,
  resave: false, saveUninitialized: false,
  cookie: { httpOnly: true, sameSite: 'strict',
    secure: false, maxAge: 15 * 60 * 1000 }
})); // secure:false is for localhost HTTP only.
```

# Server code 2/10

Append to `server.mjs` in this order.

```
const users = [
  { id: 'u1', username: 'alice', role: 'student',
    permissions: ['notes:read'],
    passwordHash: await bcrypt.hash('AliceDemo!42', 10) },
  { id: 'u2', username: 'bob', role: 'admin',
    permissions: ['notes:read', 'notes:write'],
    passwordHash: await bcrypt.hash('BobDemo!42', 10) }
];
const notes = [
  { id: 'n1', ownerId: 'u1', text: 'Alice private note' },
  { id: 'n2', ownerId: 'u2', text: 'Bob private note' }
];
const publicUser = u => ({ id: u.id, username: u.username,
  role: u.role, permissions: u.permissions });
app.get('/health', (req, res) => res.json({ status: 'ok' }));
```

# Server code 3/10

Append to `server.mjs` in this order.

```
async function checkCredentials(req, res, next) {
  const { username, password } = req.body ?? {};
  if (typeof username !== 'string' ||
      typeof password !== 'string' ||
      username.length < 1 || username.length > 40 ||
      password.length < 1 ||
      Buffer.byteLength(password, 'utf8') > 72) {
    return res.status(400).json({ error: 'Invalid input' });
  }
  const user = users.find(u => u.username === username);
  if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }
  req.user = user;
  next();
}
```

# Server code 4/10

Append to `server.mjs` in this order.

```
app.post('/session/login', checkCredentials, (req, res, next) => {
  req.session.regenerate(err => {
    if (err) return next(err);
    req.session.userId = req.user.id;
    req.session.save(err => {
      if (err) return next(err);
      res.json({ user: publicUser(req.user) });
    });
  });
});
function requireSession(req, res, next) {
  const user = users.find(u => u.id === req.session.userId);
  if (!user) return res.status(401).json({ error: 'Login required' });
  req.user = user;
  next();
}
app.get('/session/me', requireSession, (req, res) => {
  res.json({ user: publicUser(req.user) });
```

# Server code 5/10

Append to `server.mjs` in this order.

```
});
app.post('/session/logout', (req, res, next) => {
  req.session.destroy(err => {
    if (err) return next(err);
    res.clearCookie('lab.sid', { path: '/' });
    res.status(204).end();
  });
});
```

# Server code 6/10

Append to `server.mjs` in this order.

```
const tokenOptions = { algorithm: 'HS256', expiresIn: '5m',
  issuer: 'comp3028-lab', audience: 'comp3028-client' };
app.post('/jwt/login', checkCredentials, (req, res) => {
  const accessToken = jwt.sign({}, jwtSecret, {
    ...tokenOptions, subject: req.user.id
  });
  res.json({ accessToken, tokenType: 'Bearer', expiresIn: 300 });
});
function requireJwt(req, res, next) {
  const match = /^Bearer (\S+)$/.exec(req.get('Authorization') ?? '');
  if (!match) return res.status(401).json({ error: 'Bearer token required' });
  try {
    const claims = jwt.verify(match[1], jwtSecret, {
      algorithms: ['HS256'], issuer: tokenOptions.issuer,
      audience: tokenOptions.audience
    });
    const user = users.find(u => u.id === claims.sub);
    if (!user) return res.status(401).json({ error: 'Unknown user' });
```

# Server code 7/10

Append to `server.mjs` in this order.

```
req.user = user;
  } catch {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
  next();
}
app.get('/jwt/me', requireJwt, (req, res) => {
  res.json({ user: publicUser(req.user) });
});
```

# Server code 8/10

Append to `server.mjs` in this order.

```
function requireRole(role) {
  return (req, res, next) => {
    if (req.user.role !== role)
      return res.status(403).json({ error: 'Forbidden' });
    next();
  };
}
function requirePermission(permission) {
  return (req, res, next) => {
    if (!req.user.permissions.includes(permission))
      return res.status(403).json({ error: 'Forbidden' });
    next();
  };
}
app.get('/jwt/admin', requireJwt, requireRole('admin'), (req, res) => {
  res.json({ message: 'Admin access granted' });
});
app.get('/jwt/notes/:id', requireJwt,
```

# Server code 9/10

Append to `server.mjs` in this order.

```
requirePermission('notes:read'), (req, res) => {
    const note = notes.find(n => n.id === req.params.id);
    if (!note || note.ownerId !== req.user.id)
      return res.status(404).json({ error: 'Note not found' });
    res.json({ note });
  });
app.post('/jwt/notes', requireJwt,
  requirePermission('notes:write'), (req, res) => {
    const text = req.body?.text;
    if (typeof text !== 'string' || !text.trim() || text.length > 200)
      return res.status(400).json({ error: 'Text must be 1-200 characters' });
    const note = { id: `n${notes.length + 1}`,
      ownerId: req.user.id, text: text.trim() };
    notes.push(note);
    res.status(201).json({ note });
  });
app.use((err, req, res, next) => {
  const status = err.status === 400 ? 400 : err.status === 413 ? 413 : 500;
```

# Server code 10/10

Append to `server.mjs` in this order.

```
res.status(status).json({ error: status === 500 ? 'Server error' : 'Invalid request' });
});
app.listen(3000, '127.0.0.1', () => {
  console.log('API ready at http://127.0.0.1:3000');
});
```

# Original lecture resources

- Primary source: *COMP3028 Software Construction, Lecture 9: Securing APIs: Authentication and Authorisation*, supplied lecture PDF.
- Ulises Gascón, Chapter 13, "User Authentication and Authorization with Passport.js (as listed in the original lecture).
- Azat Mardan, *Practical Node.js: Building Real-World Scalable Web Apps*, Chapter 6 (as listed in the original lecture).
- "Learn JWT in 10 Minutes with Express, Node, and Cookie Parser, YouTube (original lecture suggestion; no exact video URL supplied).
- JWT debugger: `https://jwt.io/`. Use fictional example tokens only.

# Implementation references

- Express: express-session documentation. Cookie options, session regeneration, and store limitations.
- Auth0: node-jsonwebtoken documentation. Signing, verification, algorithms, issuer, audience, and expiration.
- OWASP: Password Storage Cheat Sheet. Password-hashing choices and bcrypt limits.
- IETF RFC 7519: JSON Web Token (JWT). Token claims and representation.
- OWASP: Authorization Cheat Sheet. Server-side enforcement and least privilege.

The original lecture topics are retained. Added explanations distinguish signing from encryption, decoding from validation, and role checks from object ownership.